Межпланетный манёвр

Техническая документация проекта

Версия документа: 1.0
Дата: 2026-03-21
Проект: `InterplanetaryManeuver`
Платформа: `WPF / .NET 8`
Численный метод: `Dormand–Prince RK–45`

---

1. Назначение программы

Приложение предназначено для численного моделирования гравитационного манёвра космического аппарата у Юпитера с целью получить максимальный полезный прирост скорости при дальнейшем движении в направлении Сатурна.

В текущей реализации программа решает три связанных класса задач:

1. моделирование движения системы тел в ньютоновской постановке `N-body`;
2. анализ пролёта аппарата через сферу влияния Юпитера;
3. перебор параметров сближения и выбор лучшего варианта по целевой функции.

Дополнительно в приложении реализованы:

- загрузка начальных состояний тел через `NASA/JPL Horizons API`;
- построение графиков траекторий и скоростей;
- анимация орбитальной сцены;
- песочница с произвольными телами.

---

2. Структура проекта

2.1. Основные модули

- `PhysicsSim.Core` — физическая модель, интегратор, структуры данных результата;

- `InterplanetaryManeuver.App` — WPF-приложение, UI, сценарии, оптимизация, анимация;
- `installer` и `installer-msi` — сборка установщиков.

2.2. Ключевые файлы

- `PhysicsSim.Core/NBodySystem.cs`
- `PhysicsSim.Core/NBodySimulator.cs`
- `PhysicsSim.Core/Ode/DormandPrince45.cs`
- `PhysicsSim.Core/FlybyAnalysis.cs`
- `InterplanetaryManeuver.App/Services/HorizonsEphemerisService.cs`
- `InterplanetaryManeuver.App/Services/ScenarioFactory.cs`
- `InterplanetaryManeuver.App/ViewModels/MainViewModel.cs`
- `InterplanetaryManeuver.App/Controls/Plot2DControl.cs`
- `InterplanetaryManeuver.App/Controls/OrbitSceneControl.cs`

---

3. Математическая модель

3.1. Выбор модели

В проекте используется **ньютоновская модель гравитационного взаимодействия N тел**.
Каждое тело описывается массой `m_i`, радиус-вектором `r_i(t)` и скоростью `v_i(t)`.

Состояние системы записывается как

```
y(t) = (r_1, v_1, r_2, v_2, ..., r_N, v_N)
```

где для каждого тела используются 6 компонент:

```
(x, y, z, vx, vy, vz)
```

3.2. Основные дифференциальные уравнения

Для каждого тела `i` решается система

```
r_i' = v_i
v_i' = Σ(j != i) G * m_j * (r_j - r_i) / |r_j - r_i|^3
```

где:

- `G` — гравитационная постоянная;
- `m_j` — масса тела `j`;
- `r_j - r_i` — относительный радиус-вектор;

- `|r_j - r_i|` — расстояние между телами.

Это именно та формула, которая реализована в `PhysicsSim.Core/NBodySystem.cs`.

3.3. Компонентная форма

Для координат и скоростей по осям:

```
x_i'  = vx_i
y_i'  = vy_i
z_i'  = vz_i

vx_i' = Σ(j != i) G * m_j * (x_j - x_i) / ((x_j - x_i)^2 + (y_j - y_i)^2 + (z_j
- z_i)^2)^(3/2)
vy_i' = Σ(j != i) G * m_j * (y_j - y_i) / ((x_j - x_i)^2 + (y_j - y_i)^2 + (z_j
- z_i)^2)^(3/2)
vz_i' = Σ(j != i) G * m_j * (z_j - z_i) / ((x_j - x_i)^2 + (y_j - y_i)^2 + (z_j
- z_i)^2)^(3/2)
```

3.4. Система отсчёта

По умолчанию система переводится в **барицентрическую систему отсчёта**.

Используются формулы центра масс:

```
R_cm = (Σ m_i r_i) / (Σ m_i)
V_cm = (Σ m_i v_i) / (Σ m_i)
```

После этого все начальные состояния сдвигаются:

```
r_i := r_i - R_cm
v_i := v_i - V_cm
```

Эта операция выполняется в `ShiftInitialStateToBarycentricFrame()` внутри `PhysicsSim.Core/NBodySystem.cs`.

---

4. Начальные данные и единицы измерения

4.1. Внутренние единицы

Внутри расчёта используются только единицы SI:

- расстояние — `м`;
- скорость — `м/с`;
- масса — `кг`;
- время — `с`.

4.2. Отображение в интерфейсе

В UI используются более удобные единицы:

- межпланетные расстояния — `а.е.`;
- скорости — `км/с`;
- время — `сутки` или `часы`.

4.3. Физические константы

Значения заданы в `PhysicsSim.Core/AstronomyConstants.cs`:

```
1 а.е. = 1.495978707e11 м
M⊙     = 1.98847e30 кг
M_J    = 1.89813e27 кг
M_S    = 5.6834e26 кг
R_J    = 7.1492e7 м
R_S    = 5.8232e7 м
```

---

5. Эфемериды

5.1. Что такое эфемериды

Эфемериды — это данные о положении и скорости небесного тела в конкретный момент времени.

Для данной программы это означает, что начальные состояния Солнца, Юпитера и Сатурна не задаются вручную, а загружаются из внешнего астрономического источника.

5.2. Источник данных

В программе используется `NASA/JPL Horizons API` через файл

- `InterplanetaryManeuver.App/Services/HorizonsEphemerisService.cs`

Сервис запрашивает векторы состояния (`EPHEM_TYPE='VECTORS'`) в системе `ICRF` и в единицах `км-s`, затем переводит их в SI.

5.3. Что загружается

Для каждого тела загружаются:

```
X, Y, Z, VX, VY, VZ
```

То есть полное начальное состояние в момент `epochUtc`.

5.4. Кэширование

Полученные эфемериды кэшируются локально в каталоге пользователя, чтобы уменьшить число повторных запросов.

---

6. Формирование сценария гравитационного манёвра

6.1. Базовый сценарий

Сценарий строится в `InterplanetaryManeuver.App/Services/ScenarioFactory.cs`.

В базовой задаче используются тела:

- Солнце;
- Юпитер;
- Сатурн;
- космический аппарат.

6.2. Сфера влияния Юпитера

Радиус сферы влияния вычисляется по формуле Лапласа:

```
r_SOI = a * (m_planet / m_central)^(2/5)
```

где:

- `a` — большая полуось орбиты Юпитера;
- `m_planet` — масса Юпитера;
- `m_central` — масса Солнца.

Это реализовано в `PhysicsSim.Core/FlybyAnalysis.cs`.

6.3. Начальное положение аппарата

Для аппарата вектор положения строится относительно Юпитера.

Обозначения:

- `e_r` — радиальное направление от Солнца к Юпитеру;
- `e_t` — касательное направление орбитального движения Юпитера;

- `e_n` — нормаль к плоскости, получаемая через векторное произведение.

Затем создаётся повёрнутое радиальное направление:

```
e_phase = cos(phi) * e_r + sin(phi) * e_t
```

где `phi` — фазовый угол входа.

Стартовая позиция аппарата:

```
r_sc,0 = r_J + D_start * e_phase
D_start = max(1.05, k_start) * r_SOI
```

где:

- `r_J` — положение Юпитера;
- `k_start` — множитель стартовой дистанции;
- `r_SOI` — радиус сферы влияния.

6.4. Начальная скорость аппарата

Сначала строится направление относительной скорости входа:

```
e_in = -e_phase
e_lat = e_n × e_phase
e_v = cos(alpha) * e_in + sin(alpha) * e_lat
```

где `alpha` — угол входа.

Тогда абсолютная скорость аппарата:

```
v_sc,0 = v_J + v_inf * e_v
```

где:

- `v_J` — скорость Юпитера;
- `v_inf` — гиперболическая скорость входа относительно Юпитера.

6.5. Параметры настройки пролёта

В приложении используются следующие параметры:

- `StartDistanceMultiplier`
- `PhaseAngleDeg`
- `HeadingAngleDeg`
- `VInfinityKms`

Они определены в `InterplanetaryManeuver.App/Models/FlybySetup.cs`.

7. Численный метод: Dormand-Prince RK-45

7.1. Общая идея

Интегратор находится в

- `PhysicsSim.Core/Ode/DormandPrince45.cs`

Используется адаптивная схема `RK 5(4)`:

- решение пятого порядка принимается как основное;
- решение четвёртого порядка используется для оценки локальной ошибки.

7.2. Вектор стадий

На каждом шаге вычисляются стадии

```
k1, k2, k3, k4, k5, k6, k7
```

по таблице коэффициентов Dormand–Prince.

7.3. Оценка ошибки

Если `y_new` — решение 5-го порядка, а `y_4` — вложенное решение 4-го порядка, то локальная ошибка определяется как

```
y_err = y_new - y_4
```

Нормированная RMS-ошибка:

```
scale_i = AbsTol + RelTol * max(|y_i|, |y_new_i|)
r_i     = y_err_i / scale_i
err     = sqrt( (1/n) * Σ r_i^2 )
```

Шаг принимается, если

```
err <= 1
```

7.4. Адаптация шага

Новый шаг вычисляется по формуле

```
h_new = h * clamp( safety * (1 / err)^(1/5), facMin, facMax )
```

где в коде используются:

```
safety = 0.9
facMin = 0.2
facMax = 5.0
```

7.5. Настройки интегрирования

Параметры находятся в `PhysicsSim.Core/Ode/IntegrationSettings.cs`:

- `AbsTol`
- `RelTol`
- `MinStep`
- `MaxStep`
- `InitialStep`
- `MaxAcceptedSteps`

7.6. Фиксированный вывод результатов

Хотя шаг интегрирования адаптивный, результаты сохраняются через фиксированный интервал `outputDt`.

То есть схема такая:

1. интегратор движется адаптивно;
2. но каждая выходная точка принудительно совпадает с заданным временем выборки;
3. это удобно для графиков, анимации и экспорта.

---

8. Расчёт метрик гравитационного манёвра

8.1. Минимальное расстояние до Юпитера

Для каждого дискретного момента времени вычисляется

```
r_J,sc(t) = |r_sc(t) - r_J(t)|
```

и сохраняется минимум.

8.2. Вход и выход из SOI

Считается, что аппарат вошёл в сферу влияния, если

```
|r_sc - r_J| <= r_SOI
```

Запоминаются индексы:

- `EntryIndex` — первый момент входа;
- `ExitIndex` — первый момент выхода после входа.

8.3. Скорости

Используются относительные скорости:

```
v_rel(a,b,t) = |v_a(t) - v_b(t)|
```

Из них строятся:

- начальная и конечная гелиоцентрические скорости аппарата;
- скорость аппарата относительно Юпитера до и после пролёта.

8.4. Основной прирост скорости

Полезный прирост скорости в проекте определяется так:

```
Δv_gain = |v_sc - v_sun|_(exit) - |v_sc - v_sun|_(entry)
```

То есть сравниваются именно **гелиоцентрические** скорости аппарата на входе и выходе из сферы влияния Юпитера.

Это реализовано в `PhysicsSim.Core/FlybyAnalysis.cs`.

8.5. Дополнительная метрика Сатурна

Для контроля пригодности траектории вычисляется минимальное расстояние от аппарата до Сатурна:

```
r_S,min = min_t |r_sc(t) - r_S(t)|
```

---

9. Целевая функция оптимизации

Оптимизация в приложении реализована в два этапа:

1. полный сеточный перебор (`grid search`);
2. локальная доводка лучшего кандидата по центральным разностям.

Параметры перебора:

- фазовый угол `phi`;

- угол входа `alpha`;
- гиперболическая скорость `v_inf`.

Для каждой комбинации выполняется полная симуляция и вычисляется score.

9.1. Формула score

В коде (`MainViewModel.ScoreCandidate`) используется функция

```
score = Δv_gain_kms − saturnMiss_AU − 5 * lowFlybyPenalty
```

где

```
Δv_gain_kms = Δv_gain / 1000
saturnMiss_AU = r_S,min / AU
lowFlybyPenalty = max(0, (2*R_J − r_J,min) / R_J)
```

Смысл членов функции:

- `Δv_gain_kms` — поощряет прирост скорости;
- `saturnMiss_AU` — штрафует за большой промах мимо Сатурна;
- `lowFlybyPenalty` — штрафует за слишком опасное сближение с Юпитером.

9.2. Особый случай

Если аппарат вообще не пересёк сферу влияния Юпитера, то кандидату сразу назначается большой штраф:

```
score = −1e9
```

Если аппарат столкнулся с Юпитером, то кандидату назначается ещё более жёсткий штраф:

```
score = −1e12
```

9.3. Ограничения и ранняя остановка

В текущей версии модели используются ограничения:

- столкновение с Юпитером: `r_J,min <= R_J`;
- опасно низкий пролёт: `r_J,min < 2R_J`;
- общая коллизия тел в песочнице: `d_ij <= R_i + R_j`.

При общей коллизии интегрирование останавливается в момент первого пересечения радиусов тел.

9.4. Локальная доводка

После сеточного перебора лучший допустимый кандидат можно доработать локально.

Частные производные оцениваются центральными разностями:

```
dScore/dx_i ~= (Score(x_i + h_i) - Score(x_i - h_i)) / (2h_i)
```

Для разных параметров используются разные шаги:

- для фазового угла;
- для угла входа;
- для гиперболической скорости.

---

10. Графики и анимация

10.1. Что строится

После расчёта приложение формирует:

- траектории тел в плоскости `X–Y` относительно Солнца;
- график `|v|` аппарата;
- графики компонент `Vx`, `Vy`, `Vz`;
- орбитальную анимацию.

10.2. Формулы для графиков скорости

Гелиоцентрическая скорость аппарата:

```
v_helio(t) = v_sc(t) - v_sun(t)
|v|(t) = sqrt(vx^2 + vy^2 + vz^2)
```

Компоненты на графике:

```
Vx(t), Vy(t), Vz(t)
```

10.3. Анимация

Орбитальная сцена (`OrbitSceneControl.cs`) использует:

- относительные положения тел;
- центрирование по выбранному центральному телу;

- следы траекторий;
- маркеры тел и подписи;
- шкалу кадров и регулируемую скорость воспроизведения.

---

11. Песочница пользовательских тел

В режиме песочницы пользователь может вручную задавать:

- имя тела;
- массу;
- радиус тела;
- координаты `X, Y, Z` в `a.e.`;
- скорости `Vx, Vy, Vz` в `км/с`.

Далее значения переводятся в SI:

```
x_m  = X_AU  * AU
vx_ms = Vx_kms * 1000
```

Эта логика реализована в `InterplanetaryManeuver.App/Models/EditableBody.cs`.

Если в именах присутствуют слова `Солнце`, `Юпитер`, `Сатурн`, `КА` или `spacecraft`, приложение автоматически распознаёт специальные роли тел для анализа и анимации.

---

12. Формат результата симуляции

Результат описывается классом `SimulationResult`:

```
Times        : double[]
BodyNames    : string[]
Positions    : Vector3d[][]
Velocities   : Vector3d[][]
Collision    : CollisionEvent?
TerminationReason : string?
```

Интерпретация:

```
Positions[sample][body]
Velocities[sample][body]
```

То есть сначала идёт индекс времени, затем индекс тела.

13. Основные допущения модели

В текущей версии проекта используются следующие допущения:

1. модель ньютоновская, без ОТО;
2. нет реактивной тяги и расхода массы аппарата;
3. нет солнечного давления, атмосферного торможения и нецентральных гармоник поля;
4. стартовое состояние аппарата задаётся аналитически относительно Юпитера;
5. оптимизация пока является сеточным перебором, а не непрерывным оптимизатором.

Эти допущения допустимы для учебного MVP и прозрачны для преподавателя при защите.

---

14. Что именно проверяет преподаватель

Если преподаватель будет придираться, важно акцентировать следующие моменты:

14.1. Уравнения движения реализованы явно

Они присутствуют в `PhysicsSim.Core/NBodySystem.cs` и полностью соответствуют ньютоновской постановке задачи `N-body`.

14.2. Интегратор не "магический"

В проекте не используется внешняя готовая физическая библиотека. Адаптивный `Dormand-Prince RK 5(4)` реализован вручную в `PhysicsSim.Core/Ode/DormandPrince45.cs`.

14.3. Метрика манёвра формализована

Прирост скорости определяется не «на глаз», а по строгой формуле на входе/выходе из SOI Юпитера.

14.4. Оптимизация воспроизводима

Перебор идёт по конечной сетке параметров.
Каждый кандидат можно повторить, воспроизвести и показать на графиках.

14.5. Исходные состояния тел реалистичны

Солнце, Юпитер и Сатурн берутся из `NASA/JPL Horizons API`, а не из случайных вручную введённых значений.

---

15. Примеры кода

15.1. Производные системы N тел

```
public void ComputeDerivatives(double t, ReadOnlySpan<double> y, Span<double>
dy)
{
    for (int i = 0; i < BodyCount; i++)
    {
        int baseIdx = i * 6;
        dy[baseIdx + 0] = y[baseIdx + 3];
        dy[baseIdx + 1] = y[baseIdx + 4];
        dy[baseIdx + 2] = y[baseIdx + 5];
    }

    for (int i = 0; i < BodyCount; i++)
    {
        int bi = i * 6;
        double ax = 0, ay = 0, az = 0;
        double xi = y[bi + 0];
        double yi = y[bi + 1];
        double zi = y[bi + 2];

        for (int j = 0; j < BodyCount; j++)
        {
            if (j == i) continue;
            double mj = Masses[j];
            int bj = j * 6;
            double dx = y[bj + 0] - xi;
            double dyv = y[bj + 1] - yi;
            double dz = y[bj + 2] - zi;
            double r2 = dx * dx + dyv * dyv + dz * dz;
            double invR = 1.0 / Math.Sqrt(r2);
            double invR3 = invR * invR * invR;
            double k = GravitationalConstant * mj * invR3;
            ax += k * dx;
            ay += k * dyv;
            az += k * dz;
        }

        dy[bi + 3] = ax;
        dy[bi + 4] = ay;
```

```
        dy[bi + 5] = az;
    }
}
```

15.2. Оценка шага RK-45

```
private static double ComputeErrorNorm(
    ReadOnlySpan<double> y,
    ReadOnlySpan<double> yNew,
    ReadOnlySpan<double> yErr,
    double absTol,
    double relTol)
{
    double sum = 0;
    int n = y.Length;
    for (int i = 0; i < n; i++)
    {
        double scale = absTol + relTol * Math.Max(Math.Abs(y[i]),
Math.Abs(yNew[i]));
        double r = yErr[i] / scale;
        sum += r * r;
    }
    return Math.Sqrt(sum / n);
}
```

15.3. Целевая функция оптимизации

```
private static double ScoreCandidate(FlybyMetrics metrics)
{
    if (metrics.HasJupiterCollision)
        return -1e12;

    if (!metrics.HasSphereOfInfluenceCrossing)
        return -1e9;

    double deltaVGainKms = metrics.DeltaVGainHeliocentric / 1000.0;
    double saturnMissAu = metrics.MinDistanceToSaturn /
AstronomyConstants.AstronomicalUnit;
    double lowFlybyPenalty = Math.Max(
        0.0,
        (2.0 * AstronomyConstants.JupiterMeanRadius -
metrics.MinDistanceToJupiter)
        / AstronomyConstants.JupiterMeanRadius);

    return deltaVGainKms - saturnMissAu - 5.0 * lowFlybyPenalty;
}
```

15.4. Прирост скорости на SOI Юпитера

```
double deltaVGain = RelativeSpeed(result, spacecraftIndex, sunIndex, vOutIndex)
    - RelativeSpeed(result, spacecraftIndex, sunIndex, vInIndex);
```

16. Как это можно формулировать на защите

Короткая строгая формулировка:

> В программе реализована ньютоновская N-body модель движения тел Солнечной системы. Система интегрируется адаптивным методом Dormand–Prince RK 5(4) с контролем локальной ошибки и ранней остановкой при столкновении тел. Начальные состояния Солнца, Юпитера и Сатурна загружаются через NASA/JPL Horizons API. Космический аппарат задаётся параметрами входа в сферу влияния Юпитера. После моделирования вычисляется прирост гелиоцентрической скорости аппарата на входе и выходе из SOI Юпитера, затем выполняется оптимизация: сначала сеточный перебор, затем локальная доводка лучшего кандидата по центральным разностям.

---

17. Ограничения и перспективы развития

Что можно честно сказать как ограничения текущей версии:

- пока не используется SPICE offline kernels;
- нет высокоточного учёта эллиптических элементов аппарата как отдельной задачи Ламберта;
- локальная доводка пока использует простую покоординатную схему, а не полноценный нелинейный оптимизатор;
- нет учёта ограничений на перегрузку, тепловой поток и реальные атмосферные эффекты.

Что можно назвать развитием:

- переход на SPICE kernels;
- экспорт анимации в видео;
- более интеллектуальная оптимизация;
- расширение модели миссии до нескольких гравитационных манёвров.

---

18. Вывод

Проект `InterplanetaryManeuver` реализует законченный учебно-инженерный конвейер:

1. получение реальных эфемерид;

2. формирование начальной задачи;
3. интегрирование N-body системы методом RK-45;
4. вычисление метрик гравитационного манёвра;
5. перебор параметров и выбор лучшего сценария;
6. визуализацию результата в графиках и анимации.

С точки зрения проверки преподавателем в проекте уже есть:

- строгая математическая постановка;
- прозрачная численная схема;
- воспроизводимый алгоритм оптимизации;
- явная связь формул и кода.

---

19. Список основных файлов проекта

```
PhysicsSim.Core/NBodySystem.cs
PhysicsSim.Core/NBodySimulator.cs
PhysicsSim.Core/Ode/DormandPrince45.cs
PhysicsSim.Core/FlybyAnalysis.cs
PhysicsSim.Core/FlybyMetrics.cs
PhysicsSim.Core/AstronomyConstants.cs
InterplanetaryManeuver.App/Services/HorizonsEphemerisService.cs
InterplanetaryManeuver.App/Services/ScenarioFactory.cs
InterplanetaryManeuver.App/ViewModels/MainViewModel.cs
InterplanetaryManeuver.App/Models/FlybySetup.cs
InterplanetaryManeuver.App/Models/OptimizationSettings.cs
InterplanetaryManeuver.App/Controls/OrbitSceneControl.cs
InterplanetaryManeuver.App/Controls/Plot2DControl.cs
```

---

20. Репозиторий проекта

Актуальная версия исходного кода проекта размещена в GitHub-репозитории:

`https://github.com/VladislavGubarev/InterplanetaryManeuver`

---

21. Упрощённая аналитическая модель гравиманёвра

Эта модель не заменяет численную N-body симуляцию, а используется как понятная идеализация и как проверка порядка величин.

21.1. Обозначения

- `M` — масса Юпитера;
- `G` — гравитационная постоянная;
- `R` — минимально допустимый радиус пролёта, например верхняя граница атмосферы;
- `r0` — начальное расстояние от центра Юпитера до аппарата;
- `v_p` — скорость Юпитера в гелиоцентрической системе;
- `v_inf` — гиперболическая скорость аппарата на бесконечности в системе Юпитера;
- `r_p` — перицентр траектории;
- `delta` — угол поворота асимптот гиперболы.

21.2. Энергия в системе Юпитера

Для движения аппарата в поле Юпитера:

```
v^2 / 2 - GM / r = v_inf^2 / 2
```

Для предельного идеализированного параболического случая:

```
v_inf = 0
```

Тогда на расстоянии `r0` получаем:

```
v0 = sqrt(2GM / r0)
```

21.3. Ограничение на допустимый пролёт

Траектория допустима только при условии:

```
r_p >= R
```

Если:

```
r_p < R
```

то решение считается недопустимым, так как аппарат входит в атмосферу или сталкивается с планетой.

21.4. Геометрия поворота траектории

Смысл гравиманёвра в этой модели в том, что в системе Юпитера вектор скорости аппарата поворачивается, а его модуль на бесконечности сохраняется.

В упрощённой геометрической записи используются соотношения:

```
angle(v0, v_p) = pi - arcsin(sqrt(R / r0))
angle(r0, v_p) = 2 * arcsin(sqrt(R / r0))
```

Из этих формул видно, что параметр `R` влияет на угол поворота траектории: чем больше допустимый радиус пролёта, тем шире траектория и тем слабее гравитационный разворот.

21.5. Переход в гелиоцентрическую систему

Гелиоцентрическая скорость аппарата выражается как:

```
v_hel = v_rel + v_p
```

До и после манёвра:

```
v_hel_in = v_inf_in + v_p
v_hel_out = v_inf_out + v_p
```

При идеальном gravity assist:

```
|v_inf_in| = |v_inf_out| = v_inf
```

То есть меняется в основном направление вектора скорости, а не его модуль в системе Юпитера.

21.6. Целевая функция

В упрощённой аналитической модели естественно выбирать в качестве score прирост гелиоцентрической скорости:

```
score = DeltaV = |v_hel_out| - |v_hel_in|
```

Если условие допустимости нарушено:

```
r_p < R
```

то:

```
score = -infinity
```

или решение просто исключается из оптимизации.

21.7. Физический смысл результата

- в системе Юпитера сохраняется модуль гиперболической скорости на бесконечности;
- полезный выигрыш появляется из-за поворота направления вектора скорости;

- чем ближе аппарат проходит к Юпитеру, тем больше угол поворота;
- чем больше `R`, тем слабее поворот и тем меньше итоговый прирост скорости.

21.8. Предельный идеальный случай

В предельно выгодной геометрии прирост скорости может стремиться к величине порядка:

```
DeltaV_max ~ 2 * v_p
```

Это следует понимать как верхнюю оценку для идеализированного случая, а не как обязательный результат реальной численной модели.